

Chapter 41 — Tree Dynamic Programming

0. What You Will Learn in This Chapter

Tree Dynamic Programming is used when a problem is defined on a tree and the answer for a node depends on answers from its children.

Tree DP is common when:

- the input is a binary tree
- the input is an undirected tree
- each node has choices
- parent-child relationships matter
- you need the best value in a subtree
- you need count / max / min over tree structure
- you need rerooting
- you need include/exclude decisions
- you need path values through nodes
- you need subtree aggregation

By the end of this chapter, you should understand:

- What Tree DP is
- How recursion naturally creates DP on trees
- How to define state at a tree node
- How postorder traversal supports bottom-up DP
- How to solve include/exclude tree DP
- How to solve path DP
- How to solve subtree DP
- How to solve tree diameter-style DP
- How to solve binary tree camera-style DP
- How rerooting DP works conceptually
- How to avoid parent-child cycles in undirected trees

Core problems covered in this chapter:

1. Maximum Depth of Binary Tree
2. Diameter of Binary Tree
3. Balanced Binary Tree
4. Binary Tree Maximum Path Sum
5. House Robber III
6. Longest Univalue Path
7. Count Good Nodes in Binary Tree
8. Lowest Common Ancestor, recursive-state view
9. Binary Tree Cameras
10. Distribute Coins in Binary Tree
11. Sum of Distances in Tree
12. Minimum Height Trees
13. Tree Diameter in Undirected Tree
14. Maximum Product of Splitted Binary Tree
15. Delete Nodes and Return Forest
16. Employee Importance, tree/graph DFS view
17. Unique Binary Search Trees, tree-count DP preview
18. All Possible Full Binary Trees, memoized tree construction preview

1. What Is Tree DP?

Tree DP means solving a tree problem by computing answers for subtrees and combining them at parent nodes.

For a node:

answer at node = function of answers from children

Example:

Maximum depth:

$\text{depth}(\text{node}) = 1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$

This is DP because each subtree answer is computed once and reused by the parent.

2. Why Tree DP Exists

Trees are recursive structures.

A tree is made of:

root + left subtree + right subtree

or, in a general tree:

node + children subtrees

Many tree problems ask for values that can be built from children.

Example:

Diameter of Binary Tree.

For each node, the longest path passing through it is:

$\text{leftHeight} + \text{rightHeight}$

To know that, we need child heights.

So we compute child results first.

That is postorder Tree DP.

3. Core Tree DP Idea

Most Tree DP follows this pattern:

```
State solve(node) {  
    if node is null:  
        return base state  
  
    leftState = solve(node->left)  
    rightState = solve(node->right)  
  
    currentState = combine(leftState, rightState, node)  
  
    update global answer if needed
```

```
    return currentState
}
```

The traversal is usually:

postorder

because children must be solved before the parent.

4. Tree DP vs Normal DFS

DFS only traverses.

Tree DP traverses and returns meaningful state.

Example DFS:

```
void dfs(node) {
    dfs(node->left);
    dfs(node->right);
}
```

Tree DP:

```
int dfs(node) {
    int left = dfs(node->left);
    int right = dfs(node->right);

    return 1 + max(left, right);
}
```

The return value is the DP state.

5. Tree DP State Types

5.1 Single Value State

Return one value.

Example:

```
height(node)
```

Used in:

- Maximum Depth
- Diameter
- Balanced Tree

5.2 Pair State

Return two values.

Example:

House Robber III:

```
rob = max money if rob current node
skip = max money if skip current node
```

5.3 Multi-State DP

Return multiple states.

Example:

Binary Tree Cameras:

state 0 = node has camera
state 1 = node is covered
state 2 = node is not covered

5.4 Global Answer + Return State

Return one local value, update global answer.

Example:

Binary Tree Maximum Path Sum:

return max downward path
global answer = best path anywhere

5.5 Rerooting State

Compute answer for every possible root.

Example:

Sum of Distances in Tree.

Need two passes:

19. postorder subtree DP
 20. preorder reroot DP
-

6. When to Use Tree DP

Use Tree DP when:

6.1 Problem Is on a Tree

Examples:

- binary tree
- N-ary tree
- undirected tree

6.2 Parent Answer Depends on Children

Examples:

- height
- diameter
- subtree sum
- subtree count

6.3 Need Best Choice Per Node

Examples:

- House Robber III
- Binary Tree Cameras

6.4 Need Path Through Node

Examples:

- Binary Tree Maximum Path Sum
- Diameter
- Longest Univalued Path

6.5 Need All-Node Answers

Examples:

- Sum of Distances in Tree
 - rerooting DP
-

7. When NOT to Use Tree DP

Do not force Tree DP when:

7.1 Tree Is Actually a Graph With Cycles

Use graph algorithms and visited set.

7.2 Need Shortest Path in Weighted Graph

Use Dijkstra.

7.3 Need Level Order Information

Use BFS.

Examples:

- Binary Tree Level Order Traversal
- Right Side View

7.4 Need Search Only

Use normal DFS/BFS.

Example:

Find if value exists.

7.5 Need Dynamic Updates

Use advanced data structures, not basic Tree DP.

8. Mental Models

8.1 Subtree Answer

For every node, ask:

What answer can this subtree provide to its parent?

8.2 Return vs Global Answer

Some problems need:

return value = best extendable path upward
global answer = best complete answer anywhere

This is common in path problems.

8.3 Include / Exclude

At each node:

take this node
skip this node

Used in House Robber III and independent-set style tree DP.

8.4 Covered / Not Covered

Some tree problems need state categories.

Example:

Binary Tree Cameras.

8.5 Parent Avoidance in Undirected Trees

For undirected tree DFS:

```
dfs(node, parent)
```

Avoid going back to parent.

9. Pattern Recognition Checklist

Ask:

21. Is the input a tree?
22. Can I solve children before parent?
23. What should each node return to its parent?
24. Is the answer in the root state or global?
25. Does the problem involve paths through nodes?
26. Does each node have choices?
27. Do I need include/exclude states?
28. Is the tree undirected and needs parent tracking?
29. Do I need answer for every node?
30. Is rerooting needed?

10. C++ Tools Needed

10.1 Binary Tree Node

```
struct TreeNode {  
    int val;  
    TreeNode* left;  
    TreeNode* right;  
};
```

10.2 Recursive DFS

```
int dfs(TreeNode* node) {  
    if (!node) {  
        return 0;  
    }  
  
    int left = dfs(node->left);  
    int right = dfs(node->right);  
  
    return 1 + max(left, right);  
}
```

10.3 Pair Return

```
pair<int, int> dfs(TreeNode* node) {  
    if (!node) {  
        return {0, 0};  
    }
```

```

    }

    auto left = dfs(node->left);
    auto right = dfs(node->right);

    return {state1, state2};
}

```

10.4 Undirected Tree Adjacency List

```

vector<vector<int>> graph(n);

for (auto& edge : edges) {
    int u = edge[0];
    int v = edge[1];

    graph[u].push_back(v);
    graph[v].push_back(u);
}

```

10.5 DFS With Parent

```

void dfs(int node, int parent) {
    for (int nei : graph[node]) {
        if (nei == parent) {
            continue;
        }

        dfs(nei, node);
    }
}

```

11. Core Templates

Template 1 — Basic Binary Tree DP

```

int dfs(TreeNode* node) {
    if (!node) {
        return baseValue;
    }

    int left = dfs(node->left);
    int right = dfs(node->right);
}

```

```
    return combine(node->val, left, right);  
}
```

Template 2 — Global Answer + Return State

```
int answer = 0;  
  
int dfs(TreeNode* node) {  
    if (!node) {  
        return 0;  
    }  
  
    int left = dfs(node->left);  
    int right = dfs(node->right);  
  
    answer = max(answer, useBothChildren(left, right, node->val));  
  
    return valueExtendableToParent(left, right, node->val);  
}
```

Used in:

- Diameter
- Maximum Path Sum
- Longest Univalue Path

Template 3 — Include / Exclude Tree DP

```
pair<int, int> dfs(TreeNode* node) {  
    if (!node) {  
        return {0, 0};  
    }  
  
    auto left = dfs(node->left);  
    auto right = dfs(node->right);  
  
    int take = node->val + left.second + right.second;  
    int skip = max(left.first, left.second) + max(right.first, right.second);  
  
    return {take, skip};  
}
```

Used in:

- House Robber III

Template 4 — Undirected Tree Postorder DP

```

void dfs(int node, int parent) {
    for (int nei : graph[node]) {
        if (nei == parent) {
            continue;
        }

        dfs(nei, node);

        // combine child info into node
    }
}

```

Template 5 — Rerooting DP Two Passes

```

void postorder(int node, int parent) {
    count[node] = 1;

    for (int nei : graph[node]) {
        if (nei == parent) {
            continue;
        }

        postorder(nei, node);

        count[node] += count[nei];
        answer[node] += answer[nei] + count[nei];
    }
}

void preorder(int node, int parent) {
    for (int nei : graph[node]) {
        if (nei == parent) {
            continue;
        }

        answer[nei] = answer[node] - count[nei] + (n - count[nei]);

        preorder(nei, node);
    }
}

```

Used in:

- Sum of Distances in Tree

12. Major Tree DP Variations

Height DP	height of subtree	Max Depth
Diameter DP	return height, update diameter	Diameter
Path DP	return downward path, update global path	Max Path Sum
Include/exclude	take/skip node	House Robber III
Subtree balance	height or invalid marker	Balanced Tree
Camera states	covered/uncovered/camera	Binary Tree Cameras
Flow/balance	surplus returned to parent	Distribute Coins
Subtree count/sum	count and distance	Sum of Distances
Rerooting	answer for every root	Sum of Distances
Tree construction memo	memo by node count	Full Binary Trees

13. Full Problem Walkthrough 1 — LeetCode 104: Maximum Depth of Binary Tree

13.1 Problem Summary

Given binary tree, return maximum depth.

13.2 State

$\text{dfs}(\text{node})$ = maximum depth of subtree rooted at node

13.3 Transition

$\text{depth}(\text{node}) = 1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$

13.4 C++ Code

```

class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (!root) {
            return 0;
        }

        int left = maxDepth(root->left);
        int right = maxDepth(root->right);

        return 1 + max(left, right);
    }
};

```

13.5 Complexity

Time:

$O(n)$

Space:

$O(h)$

where h is tree height due to recursion stack.

14. Full Problem Walkthrough 2 — LeetCode 543: Diameter of Binary Tree

14.1 Problem Summary

Return length of longest path between any two nodes.
Length is measured in edges.

14.2 State

Return:

height(node)

Global answer:

diameter = max(leftHeight + rightHeight)

14.3 Why Global Answer?

The longest path may pass through any node, not necessarily root.

14.4 C++ Code

```

class Solution {
public:

```

```
int diameter = 0;

int height(TreeNode* node) {
    if (!node) {
        return 0;
    }

    int left = height(node->left);
    int right = height(node->right);

    diameter = max(diameter, left + right);

    return 1 + max(left, right);
}

int diameterOfBinaryTree(TreeNode* root) {
    height(root);
    return diameter;
}
};
```

14.5 Complexity

Time:

$O(n)$

Space:

$O(h)$

15. Full Problem Walkthrough 3 — LeetCode 110: Balanced Binary Tree

15.1 Problem Summary

A binary tree is height-balanced if every node has left/right subtree heights differing by at most 1.

Return true if balanced.

15.2 State

Return height if subtree is balanced.

Return -1 if subtree is not balanced.

15.3 C++ Code

```

class Solution {
public:
    int check(TreeNode* node) {
        if (!node) {
            return 0;
        }

        int left = check(node->left);

        if (left == -1) {
            return -1;
        }

        int right = check(node->right);

        if (right == -1) {
            return -1;
        }

        if (abs(left - right) > 1) {
            return -1;
        }

        return 1 + max(left, right);
    }

    bool isBalanced(TreeNode* root) {
        return check(root) != -1;
    }
};

```

15.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

15.5 Key Lesson

Returning a special invalid state is common in Tree DP.

16. Full Problem Walkthrough 4 —

LeetCode 124: Binary Tree Maximum Path Sum

16.1 Problem Summary

Find maximum path sum in binary tree.

A path can start and end at any nodes, but must follow parent-child links.

16.2 State

Return:

maximum downward path sum starting at node

Global answer:

best path anywhere

16.3 Important Distinction

A path returned to parent can use only one side:

node + left
or node + right

But global answer can use both sides:

left + node + right

16.4 C++ Code

```
class Solution {
public:
    int best = INT_MIN;

    int dfs(TreeNode* node) {
        if (!node) {
            return 0;
        }

        int left = max(0, dfs(node->left));
        int right = max(0, dfs(node->right));

        best = max(best, node->val + left + right);

        return node->val + max(left, right);
    }

    int maxPathSum(TreeNode* root) {
        dfs(root);
    }
};
```

```
        return best;
    }
};
```

16.5 Complexity

Time:

$O(n)$

Space:

$O(h)$

16.6 Common Mistake

Returning a path using both left and right to the parent is invalid.
The parent can extend only one branch.

17. Full Problem Walkthrough 5 — LeetCode 337: House Robber III

17.1 Problem Summary

Rob houses arranged as a binary tree.
If you rob a node, you cannot rob its children.
Return maximum money.

17.2 State

For each node:

take = max money if rob this node
skip = max money if skip this node

17.3 Transition

If take node:

take = node->val + left.skip + right.skip

If skip node:

skip = max(left.take, left.skip) + max(right.take, right.skip)

17.4 C++ Code

```
class Solution {
public:
    pair<int, int> dfs(TreeNode* node) {
        if (!node) {
            return {0, 0};
        }

        auto left = dfs(node->left);
        auto right = dfs(node->right);
```

```

    int take = node->val + left.second + right.second;

    int skip = max(left.first, left.second)
        + max(right.first, right.second);

    return {take, skip};
}

int rob(TreeNode* root) {
    auto result = dfs(root);

    return max(result.first, result.second);
}
};

```

17.5 Complexity

Time:

$O(n)$

Space:

$O(h)$

18. Full Problem Walkthrough 6 — LeetCode 687: Longest Univalue Path

18.1 Problem Summary

Find longest path where every node has the same value.
Return path length in edges.

18.2 State

Return:

longest downward univalue path from node

Global answer:

best leftArrow + rightArrow

18.3 C++ Code

```

class Solution {
public:
    int best = 0;

```

```

int dfs(TreeNode* node) {
    if (!node) {
        return 0;
    }

    int left = dfs(node->left);
    int right = dfs(node->right);

    int leftArrow = 0;
    int rightArrow = 0;

    if (node->left && node->left->val == node->val) {
        leftArrow = left + 1;
    }

    if (node->right && node->right->val == node->val) {
        rightArrow = right + 1;
    }

    best = max(best, leftArrow + rightArrow);

    return max(leftArrow, rightArrow);
}

int longestUnivaluePath(TreeNode* root) {
    dfs(root);
    return best;
}
};

```

18.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

19. Full Problem Walkthrough 7 — LeetCode 1448: Count Good Nodes in Binary Tree

19.1 Problem Summary

A node is good if no ancestor has value greater than it.
Return number of good nodes.

19.2 State

This is more DFS with path state than bottom-up DP.
Pass down:

max value seen so far

19.3 C++ Code

```
class Solution {
public:
    int dfs(TreeNode* node, int maxSoFar) {
        if (!node) {
            return 0;
        }

        int good = 0;

        if (node->val >= maxSoFar) {
            good = 1;
        }

        int nextMax = max(maxSoFar, node->val);

        return good
            + dfs(node->left, nextMax)
            + dfs(node->right, nextMax);
    }

    int goodNodes(TreeNode* root) {
        return dfs(root, INT_MIN);
    }
};
```

19.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

19.5 Lesson

Not all tree problems are bottom-up.
Some use top-down state from ancestors.

20. Full Problem Walkthrough 8 — Lowest Common Ancestor, Recursive-State View

20.1 Problem Summary

Given binary tree and two nodes p and q .
Return their lowest common ancestor.

20.2 Recursive State

Each subtree returns:

p if p found
 q if q found
LCA if both found
null if neither found

20.3 C++ Code

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p,
    TreeNode* q) {
        if (!root || root == p || root == q) {
            return root;
        }

        TreeNode* left = lowestCommonAncestor(root->left, p, q);
        TreeNode* right = lowestCommonAncestor(root->right, p, q);

        if (left && right) {
            return root;
        }

        return left ? left : right;
    }
};
```

20.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

21. Full Problem Walkthrough 9 — LeetCode 968: Binary Tree Cameras

21.1 Problem Summary

Place cameras on nodes.

A camera covers:

parent
itself
children

Return minimum cameras needed to cover all nodes.

21.2 Greedy Tree DP State

Use states:

0 = node has camera
1 = node is covered but has no camera
2 = node is not covered

Process children first.

Rules:

- If any child is not covered, place camera here.
- Else if any child has camera, this node is covered.
- Else this node is not covered.

21.3 C++ Code

```
class Solution {
public:
    int cameras = 0;

    int dfs(TreeNode* node) {
        if (!node) {
            return 1;
        }

        int left = dfs(node->left);
```

```

    int right = dfs(node->right);

    if (left == 2 || right == 2) {
        cameras++;
        return 0;
    }

    if (left == 0 || right == 0) {
        return 1;
    }

    return 2;
}

int minCameraCover(TreeNode* root) {
    if (dfs(root) == 2) {
        cameras++;
    }

    return cameras;
}
};

```

21.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

21.5 Key Lesson

Tree DP often uses categorical states.

22. Full Problem Walkthrough 10 — LeetCode 979: Distribute Coins in Binary Tree

22.1 Problem Summary

Each node has coins.

There are n nodes and n coins total.

Move one coin along one edge costs 1 move.

Return minimum moves so every node has exactly one coin.

22.2 State

Return surplus coins from subtree:

surplus = coins in subtree - nodes in subtree

If surplus positive, send coins up.

If negative, need coins from parent.

Moves added:

$\text{abs}(\text{leftSurplus}) + \text{abs}(\text{rightSurplus})$

22.3 C++ Code

```
class Solution {
public:
    int moves = 0;

    int dfs(TreeNode* node) {
        if (!node) {
            return 0;
        }

        int left = dfs(node->left);
        int right = dfs(node->right);

        moves += abs(left) + abs(right);

        return node->val + left + right - 1;
    }

    int distributeCoins(TreeNode* root) {
        dfs(root);
        return moves;
    }
};
```

22.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

23. Full Problem Walkthrough 11 — LeetCode 834: Sum of Distances in Tree

23.1 Problem Summary

Given an undirected tree with n nodes.

For each node, return sum of distances from that node to all other nodes.

23.2 Why This Needs Rerooting DP

If we compute distance sum from every node separately, it is $O(n^2)$.

Rerooting computes all answers in $O(n)$.

23.3 State

First pass:

count[node] = number of nodes in subtree

answer[node] = sum of distances from node to nodes in its subtree

Second pass reroots answer from parent to child.

23.4 Reroot Formula

When rerooting from node to child nei:

Nodes in child's subtree get 1 closer:

-count[nei]

All other nodes get 1 farther:

+(n - count[nei])

So:

answer[nei] = answer[node] - count[nei] + (n - count[nei])

23.5 C++ Code

```
class Solution {
public:
    vector<vector<int>>> graph;
    vector<int> count;
    vector<int> answer;
    int n;

    void postorder(int node, int parent) {
        count[node] = 1;

        for (int nei : graph[node]) {
            if (nei == parent) {
                continue;
            }
        }
    }
};
```

```

        postorder(nei, node);

        count[node] += count[nei];
        answer[node] += answer[nei] + count[nei];
    }
}

void preorder(int node, int parent) {
    for (int nei : graph[node]) {
        if (nei == parent) {
            continue;
        }

        answer[nei] = answer[node] - count[nei] + (n - count[nei]);

        preorder(nei, node);
    }
}

vector<int> sumOfDistancesInTree(int N, vector<vector<int>>& edges) {
    n = N;

    graph.assign(n, {});
    count.assign(n, 0);
    answer.assign(n, 0);

    for (auto& edge : edges) {
        int u = edge[0];
        int v = edge[1];

        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    postorder(0, -1);
    preorder(0, -1);

    return answer;
}
};

```

23.6 Complexity

Time:

$O(n)$

Space:

$O(n)$

23.7 Key Lesson

Rerooting DP computes answers for every possible root.

24. Full Problem Walkthrough 12 — Minimum Height Trees

24.1 Problem Summary

Given an undirected tree.

Return all roots that produce minimum tree height.

24.2 Tree DP / Graph Insight

The roots of minimum height trees are tree centers.

They can be found by repeatedly removing leaves.

This is not classic DP, but related to tree structure optimization.

24.3 C++ Code

```
class Solution {
public:
    vector<int> findMinHeightTrees(int n, vector<vector<int>>& edges) {
        if (n == 1) {
            return {0};
        }

        vector<vector<int>> graph(n);
        vector<int> degree(n, 0);

        for (auto& edge : edges) {
            int u = edge[0];
            int v = edge[1];

            graph[u].push_back(v);
            graph[v].push_back(u);

            degree[u]++;
            degree[v]++;
        }

        queue<int> q;
```

```

for (int i = 0; i < n; i++) {
    if (degree[i] == 1) {
        q.push(i);
    }
}

int remaining = n;

while (remaining > 2) {
    int size = q.size();
    remaining -= size;

    while (size-- > 0) {
        int leaf = q.front();
        q.pop();

        for (int nei : graph[leaf]) {
            degree[nei]--;

            if (degree[nei] == 1) {
                q.push(nei);
            }
        }
    }
}

vector<int> result;

while (!q.empty()) {
    result.push_back(q.front());
    q.pop();
}

return result;
};

```

24.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

25. Full Problem Walkthrough 13 — Tree Diameter in Undirected Tree

25.1 Problem Summary

Given undirected tree, return diameter length.

25.2 DP Approach

Root the tree anywhere.

For each node, compute two largest child heights.

Diameter through node:

largestHeight + secondLargestHeight

25.3 C++ Code

```
int treeDiameter(int n, vector<vector<int>>& edges) {
    vector<vector<int>> graph(n);

    for (auto& edge : edges) {
        int u = edge[0];
        int v = edge[1];

        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    int diameter = 0;

    function<int(int, int)> dfs = [&](int node, int parent) {
        int best1 = 0;
        int best2 = 0;

        for (int nei : graph[node]) {
            if (nei == parent) {
                continue;
            }

            int h = 1 + dfs(nei, node);

            if (h > best1) {
                best2 = best1;
                best1 = h;
            }
        }

        return best1;
    };

    dfs(0, -1);

    return diameter;
}
```

```

        } else if (h > best2) {
            best2 = h;
        }
    }

    diameter = max(diameter, best1 + best2);

    return best1;
};

dfs(0, -1);

return diameter;
}

```

25.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

26. Full Problem Walkthrough 14 — Maximum Product of Splitted Binary Tree

26.1 Problem Summary

Remove one edge to split binary tree into two subtrees.
Return maximum product of sums of the two parts.

26.2 State

First compute total tree sum.

Then during second DFS, each subtree sum gives split product:

```
subtreeSum * (totalSum - subtreeSum)
```

26.3 C++ Code

```

class Solution {
public:
    long long total = 0;
    long long best = 0;
    const int MOD = 1000000007;

```

```

long long getTotal(TreeNode* node) {
    if (!node) {
        return 0;
    }

    return node->val + getTotal(node->left) + getTotal(node->right);
}

long long dfs(TreeNode* node) {
    if (!node) {
        return 0;
    }

    long long sum = node->val + dfs(node->left) + dfs(node->right);

    best = max(best, sum * (total - sum));

    return sum;
}

int maxProduct(TreeNode* root) {
    total = getTotal(root);

    dfs(root);

    return best % MOD;
}
};

```

26.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

27. Full Problem Walkthrough 15 — Delete Nodes and Return Forest

27.1 Problem Summary

Given binary tree and list of values to delete.

Delete those nodes and return roots of resulting forest.

27.2 Tree DFS State

Return updated subtree root after deletion.

If current node is deleted, its non-deleted children become new roots.

27.3 C++ Code

```
class Solution {
public:
    unordered_set<int> deleteSet;
    vector<TreeNode*> forest;

    TreeNode* dfs(TreeNode* node, bool isRoot) {
        if (!node) {
            return nullptr;
        }

        bool deleted = deleteSet.count(node->val);

        if (isRoot && !deleted) {
            forest.push_back(node);
        }

        node->left = dfs(node->left, deleted);
        node->right = dfs(node->right, deleted);

        if (deleted) {
            return nullptr;
        }

        return node;
    }

    vector<TreeNode*> delNodes(TreeNode* root, vector<int>& to_delete) {
        for (int x : to_delete) {
            deleteSet.insert(x);
        }

        dfs(root, true);

        return forest;
    }
};
```

27.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

28. Full Problem Walkthrough 16 — Employee Importance

28.1 Problem Summary

Each employee has importance value and subordinates.

Given employee id, return total importance of that employee and all subordinates.

28.2 Tree / Graph DFS View

This is subtree sum over an employee hierarchy.

If hierarchy is guaranteed tree-like, simple DFS works.

28.3 C++ Code

```
class Solution {
public:
    unordered_map<int, Employee*> mp;

    int dfs(int id) {
        Employee* employee = mp[id];

        int total = employee->importance;

        for (int sub : employee->subordinates) {
            total += dfs(sub);
        }

        return total;
    }

    int getImportance(vector<Employee*> employees, int id) {
        for (Employee* employee : employees) {
            mp[employee->id] = employee;
        }

        return dfs(id);
    }
};
```

28.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

29. Full Problem Walkthrough 17 — Unique Binary Search Trees, Tree-Count DP Preview

29.1 Problem Summary

Given n , count structurally unique BSTs storing values $1..n$.

29.2 Why This Relates to Tree DP

Choose each value as root.

Left subtree has root - 1 nodes.

Right subtree has n - root nodes.

29.3 State

$dp[n]$ = number of unique BSTs with n nodes

29.4 Transition

$dp[n] = \text{sum}(dp[\text{leftSize}] * dp[\text{rightSize}])$

29.5 C++ Code

```
class Solution {
public:
    int numTrees(int n) {
        vector<int> dp(n + 1, 0);

        dp[0] = 1;
        dp[1] = 1;

        for (int nodes = 2; nodes <= n; nodes++) {
            for (int left = 0; left < nodes; left++) {
                int right = nodes - 1 - left;

                dp[nodes] += dp[left] * dp[right];
            }
        }

        return dp[n];
    }
};
```

```
}  
};
```

29.6 Complexity

Time:

$O(n^2)$

Space:

$O(n)$

30. Full Problem Walkthrough 18 — All Possible Full Binary Trees, Memoized Tree Construction Preview

30.1 Problem Summary

Return all full binary trees with n nodes.

A full binary tree has every node with either 0 or 2 children.

30.2 State

$\text{memo}[n]$ = all full binary trees with n nodes

Only odd n is possible.

30.3 High-Level Idea

Choose left subtree size and right subtree size:

$\text{left} + \text{right} + 1 = n$

Combine every left tree with every right tree.

30.4 C++ Code

```
class Solution {  
public:  
    unordered_map<int, vector<TreeNode*>> memo;  
  
    vector<TreeNode*> allPossibleFBT(int n) {  
        if (memo.count(n)) {  
            return memo[n];  
        }  
  
        vector<TreeNode*> result;  
  
        if (n == 1) {
```

```

        result.push_back(new TreeNode(0));
        memo[n] = result;
        return result;
    }

    if (n % 2 == 0) {
        memo[n] = result;
        return result;
    }

    for (int leftSize = 1; leftSize < n; leftSize += 2) {
        int rightSize = n - 1 - leftSize;

        vector<TreeNode*> leftTrees = allPossibleFBT(leftSize);
        vector<TreeNode*> rightTrees = allPossibleFBT(rightSize);

        for (TreeNode* left : leftTrees) {
            for (TreeNode* right : rightTrees) {
                TreeNode* root = new TreeNode(0);
                root->left = left;
                root->right = right;

                result.push_back(root);
            }
        }
    }

    memo[n] = result;
    return result;
}
};

```

30.5 Lesson

Tree DP can count or construct tree structures.

31. Common Tree DP Edge Cases

31.1 Empty Tree

Often return:

0
null
true

depending on problem.

31.2 Single Node

Check:

- depth
- diameter
- camera
- path sum
- robber

31.3 Negative Values

Important for maximum path sum.

Initialize answer carefully:

```
INT_MIN
```

31.4 Skewed Tree

Recursion depth becomes $O(n)$.

May cause stack issues in some environments.

31.5 Undirected Tree Parent

Always avoid revisiting parent:

```
if (nei == parent) continue;
```

31.6 Global Answer Reset

If using class variables, make sure they are initialized per call.

32. Common Tree DP Mistakes

Mistake 1 — Returning the Wrong State

In maximum path sum, return one branch upward, not both branches.

Mistake 2 — Forgetting Global Answer

Diameter and max path can occur below the root.

Mistake 3 — Confusing Node Count and Edge Count

Diameter may be measured in edges or nodes.

LeetCode Diameter of Binary Tree returns edges.

Mistake 4 — Not Handling Negative Paths

For max path sum, discard negative child paths with:

```
max(0, dfs(child))
```

Mistake 5 — Infinite DFS in Undirected Tree

Always pass parent or visited set.

Mistake 6 — Wrong Null State

For Binary Tree Cameras, null should be considered covered.

33. Interview Explanation Templates

33.1 Basic Tree DP

I solve the tree bottom-up. For each node, I first compute the DP states for its children, then combine those states to compute the state for the current node.

33.2 Diameter

For each node, I compute the height of its left and right subtrees. The path through that node has length left height plus right height, so I update a global maximum.

33.3 Maximum Path Sum

The value returned to the parent is the best downward path using one branch. Separately, I update the global answer using both left and right branches through the current node.

33.4 House Robber III

For each node, I maintain two states: the best result if I rob this node and the best result if I skip it. Robbing this node forces skipping its children.

33.5 Rerooting

I first compute subtree information with a postorder DFS. Then I do a second DFS to transfer the parent's answer to each child as if the child became the root.

34. Complexity Patterns

Basic binary tree DP	$O(n)$	$O(h)$
Diameter	$O(n)$	$O(h)$
Maximum path sum	$O(n)$	$O(h)$
House Robber III	$O(n)$	$O(h)$
Binary Tree Cameras	$O(n)$	$O(h)$
Distribute Coins	$O(n)$	$O(h)$
Undirected tree DP	$O(n)$	$O(n)$
Rerooting DP	$O(n)$	$O(n)$
Tree construction DP	output-dependent	output-dependent

35. Time-Boxed Practice Guidance

Intro Tree DP

Target:

20–35 minutes

Problems:

- Maximum Depth
- Balanced Binary Tree
- Diameter of Binary Tree

Medium Tree DP

Target:

35–60 minutes

Problems:

- Binary Tree Maximum Path Sum
- House Robber III
- Longest Univalued Path
- Distribute Coins

Hard Tree DP

Target:

60–90 minutes

Problems:

- Binary Tree Cameras
- Sum of Distances in Tree
- Maximum Product of Splitted Binary Tree
- All Possible Full Binary Trees

36. Expanded Practice Set

36.1 Must-Do Easy / Medium

31. Maximum Depth of Binary Tree
32. Diameter of Binary Tree
33. Balanced Binary Tree
34. Count Good Nodes in Binary Tree
35. Lowest Common Ancestor
36. Longest Univalued Path
37. House Robber III
38. Binary Tree Maximum Path Sum
39. Distribute Coins in Binary Tree

36.2 Strong Interview Practice

40. Binary Tree Cameras
41. Maximum Product of Splitted Binary Tree
42. Delete Nodes and Return Forest
43. Sum of Distances in Tree
44. Minimum Height Trees
45. Tree Diameter in Undirected Tree
46. Employee Importance
47. Unique Binary Search Trees

36.3 Hard / Stretch

48. Sum of Distances in Tree
49. Binary Tree Cameras
50. All Possible Full Binary Trees
51. Number of Ways to Reorder Array to Get Same BST
52. Count Nodes With the Highest Score
53. Tree of Coprimes
54. Minimum Edge Reversals So Every Node Is Reachable

37. Suggested Study Order

Step 1 — Basic Subtree Returns

Start with:

- 55. Maximum Depth
- 56. Balanced Binary Tree
- 57. Diameter

Goal:

Learn postorder state returns.

Step 2 — Global Answer Path DP

Solve:

- 58. Binary Tree Maximum Path Sum
- 59. Longest Univalue Path

Goal:

Distinguish return value from global answer.

Step 3 — Include / Exclude States

Solve:

- 60. House Robber III
- 61. Binary Tree Cameras

Goal:

Learn multi-state node DP.

Step 4 — Subtree Flow / Sums

Solve:

- 62. Distribute Coins
- 63. Maximum Product of Splitted Binary Tree

Goal:

Learn subtree balance and sum returns.

Step 5 — Undirected Tree DP

Solve:

- 64. Tree Diameter
- 65. Sum of Distances in Tree
- 66. Minimum Height Trees

Goal:

Learn parent tracking and rerooting.

38. Pattern Decision Table

max depth / height	return subtree height	Max Depth
longest path edges	return height + global best	Diameter
max sum path anywhere	return downward path + global best	Max Path Sum

Problem Clue	Tree DP Strategy	Example
cannot choose parent and child	include/exclude states	House Robber III
camera coverage	categorical states	Binary Tree Cameras
coins move through edges	return subtree surplus	Distribute Coins
split by edge	subtree sums	Max Product Split
all roots answer	rerooting DP	Sum of Distances
undirected tree path	DFS with parent	Tree Diameter
construct all trees	memo by node count	Full Binary Trees

39. Mini Problem 1 — Subtree Sum

```
int subtreeSum(TreeNode* node) {
    if (!node) {
        return 0;
    }

    int left = subtreeSum(node->left);
    int right = subtreeSum(node->right);

    return node->val + left + right;
}
```

State:

sum(node) = sum of values in subtree rooted at node

40. Mini Problem 2 — Count Nodes in Each Subtree, Undirected Tree

```
vector<int> subtreeSize;
vector<vector<int>> graph;

void dfs(int node, int parent) {
    subtreeSize[node] = 1;
```

```

for (int nei : graph[node]) {
    if (nei == parent) {
        continue;
    }

    dfs(nei, node);

    subtreeSize[node] += subtreeSize[nei];
}
}

```

41. Mini Problem 3 — Tree Height in Undirected Tree

```

int dfsHeight(int node, int parent, vector<vector<int>>& graph) {
    int best = 0;

    for (int nei : graph[node]) {
        if (nei == parent) {
            continue;
        }

        best = max(best, 1 + dfsHeight(nei, node, graph));
    }

    return best;
}

```

42. Mini Problem 4 — Generic Pair State

```

pair<int, int> dfs(TreeNode* node) {
    if (!node) {
        return {0, 0};
    }

    auto left = dfs(node->left);
    auto right = dfs(node->right);
}

```

```
int stateA = combineA(node, left, right);
int stateB = combineB(node, left, right);

return {stateA, stateB};
}
```

Used when each node has two choices.

43. Common Edge Case Checklist

Before submitting Tree DP code, test:

empty tree
single node
only left child chain
only right child chain
balanced tree
all negative values
duplicate values
root is answer
answer is in lower subtree
undirected tree with $n = 1$
star-shaped tree
line-shaped tree

44. Debugging Tree DP

If Tree DP fails:

67. Define exactly what `dfs(node)` returns.
68. Check null base case.
69. Check whether answer should be returned or global.
70. Check whether path can use one branch or both.
71. Check edge count vs node count.
72. Check negative values.
73. For undirected trees, check parent handling.
74. For rerooting, verify both DFS passes.
75. Test a single-node tree.

45. What to Master Before Chapter 42

Before moving to Digit Dynamic Programming, make sure you can:

- Write postorder Tree DP
- Define state returned by a node
- Solve Maximum Depth

- Solve Diameter
 - Solve Balanced Binary Tree
 - Solve Binary Tree Maximum Path Sum
 - Solve House Robber III
 - Solve Binary Tree Cameras
 - Solve Distribute Coins
 - Understand global answer vs return state
 - Handle undirected tree DFS with parent
 - Understand rerooting DP basics
 - Solve Sum of Distances in Tree conceptually
-

46. Final Cheat Sheet

Basic Height

```
return 1 + max(dfs(node->left), dfs(node->right));
```

Diameter

```
diameter = max(diameter, leftHeight + rightHeight);
return 1 + max(leftHeight, rightHeight);
```

Maximum Path Sum

```
left = max(0, dfs(node->left));
right = max(0, dfs(node->right));

best = max(best, node->val + left + right);

return node->val + max(left, right);
```

House Robber III

```
take = node->val + left.skip + right.skip;
skip = max(left.take, left.skip) + max(right.take, right.skip);
```

Distribute Coins

```
moves += abs(left) + abs(right);
return node->val + left + right - 1;
```

Rerooting Distance

```
answer[child] = answer[parent] - count[child] + (n - count[child]);
```

47. Core Takeaway

Tree DP is recursion with meaningful return values.
For every node, ask:

What information does this node need from its children?
What information should this node return to its parent?

Most Tree DP is postorder:

left child
right child
current node

The biggest distinction is:

return state = what parent can use
global answer = best answer anywhere

For undirected trees, always avoid revisiting the parent.

For all-node answers, use rerooting DP.

Once you can clearly define the node state, Tree DP becomes one of the most natural DP patterns.

The sequence so far is:

34	Dynamic Programming Foundations	Completed
35	1D Dynamic Programming	Completed
36	2D / Grid Dynamic Programming	Completed
37	String Dynamic Programming	Completed
38	Knapsack Dynamic Programming	Completed
39	Interval Dynamic Programming	Completed
40	Bitmask Dynamic Programming	Completed
41	Tree Dynamic Programming	Completed

So **Chapter 40 = Bitmask Dynamic Programming**.

If you want, I can either:

76. **Repeat Chapter 40 — Bitmask Dynamic Programming**, or
77. Continue with **Chapter 42 — Digit Dynamic Programming**.